

Compiler Design.

Test.

Kamalesh.M

1924121438

CSA14107

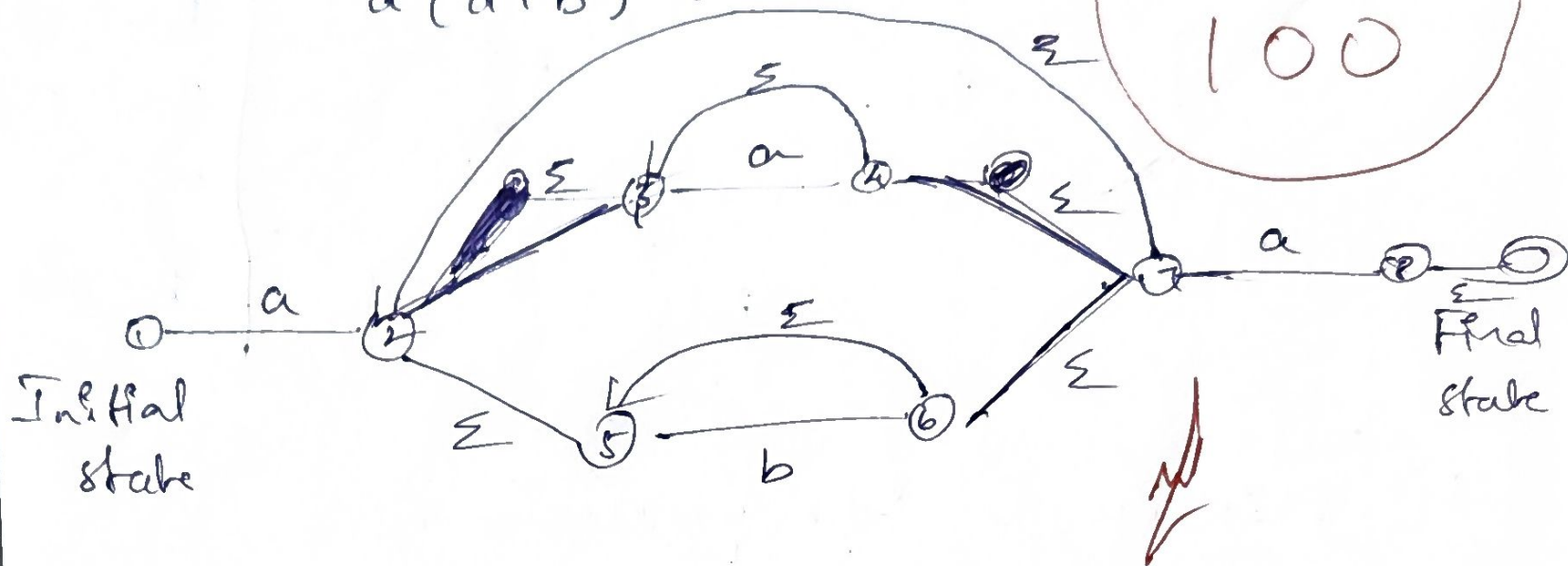
2)

as NFA for regular expression :-

$a(a|b)^*a$.

68

100



Accepting states

aa

aaa

aba

aaaa

abba

aaaaa

abbaa

Rejected states

ϵ

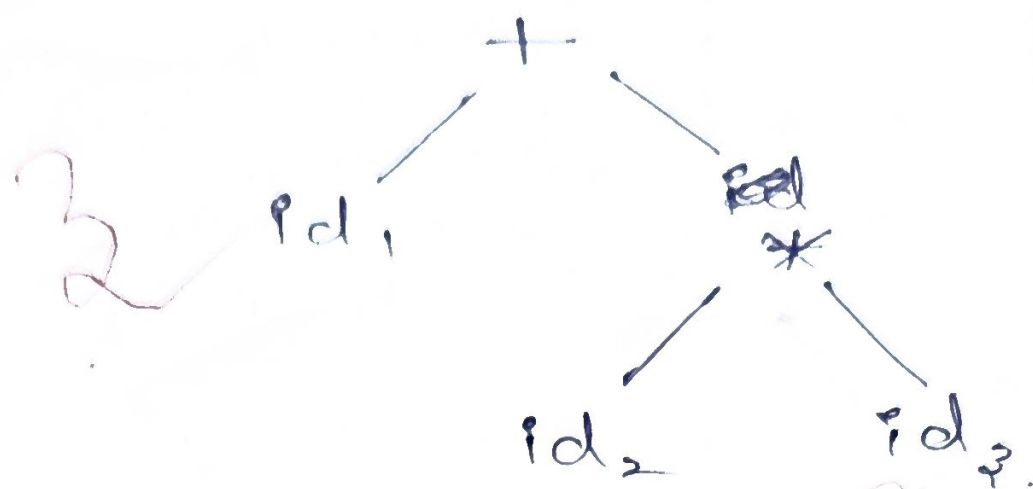
a

b

ba

5) $id + id * id$ Construct Parse tree

$\langle id_1 \rangle \langle + \rangle \langle id_2 \rangle * \langle id_3 \rangle$



$E \rightarrow TE'$

$E' \rightarrow +TE' / \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' / \epsilon$

$F \rightarrow (E) / id$

4) Eliminate left recursion

$A \rightarrow A\alpha / B$

Sol:

3 $A \rightarrow B$

$A \rightarrow \alpha A' / \epsilon$

$A \rightarrow BA'$

6) Find FIRST & FOLLOW:

$S \rightarrow aBDh$

$B \rightarrow cC$

$C \rightarrow bC / \epsilon$

$D \rightarrow EF$

$E \rightarrow g / \epsilon$

$F \rightarrow f / \epsilon$

$\text{FIRST}(A) = \{a\}$
 $\text{FIRST}(B) = \{c\}$
 $\text{FIRST}(C) = \{b\}$
 $\text{FIRST}(D) = \{q, s\}$
 $\text{FIRST}(E) = \{q, \epsilon\}$
 $\text{FIRST}(F) = \{f\}$

$\text{FOLLOW}(A) = \{\$ \}$
 $\text{FOLLOW}(B) = \{q, \$ \}$
 $\text{FOLLOW}(C) = \{b, \$ \}$
 $\text{FOLLOW}(D) = \{w, \$ \}$
 $\text{FOLLOW}(E) = \{f, \$ \}$
 $\text{FOLLOW}(F) = \{f, \$ \}$

Ex

$S \rightarrow \underset{\alpha}{PE} \underset{\beta}{TS} / \underset{\alpha}{PE} \underset{\beta}{TS} \underset{\beta}{es}$
left Factoring

$A \rightarrow \alpha R_1 / \alpha R_2$

$S \rightarrow PE TS$

$S \rightarrow PE TS es$

$S \rightarrow a.$

3

8)

 $E \rightarrow TE', E \rightarrow +TE'$

$$\text{FIRST}(E') = \{ + \}$$

$$\text{FIRST}(E) = \{ +, \epsilon \}$$

9)

$S \rightarrow (S)S \mid \epsilon$, check whether string is ...

10)

Actions Performed in Shift reduce Parser

- * $\$$ is first
- * Put the input string.
- * According to the first production take and write in reverse order.
- * We need to cancel whole input with shift action.
- * And reduce action.
- * Finally accept the input string.
- * Or reject the string.

11)

$$S' \rightarrow S$$

$$S \rightarrow CC$$

$$C \rightarrow c \mid d$$

$$\text{Item: } [S' \rightarrow \cdot S]$$

Closure:-

$S' \rightarrow \cdot S$

$S \rightarrow \cdot c \mid c \cdot$

$c \rightarrow \cdot c \mid c \cdot d$

Go to:- $(I_0, S') \in I_1$

$S' \rightarrow S \cdot$ ✓

Go to:- $(I_0, S) \in I_2$

$S \rightarrow c \cdot$

Go to:- $(I_0, c) \in I_3$

$c \rightarrow c \cdot$

$c \rightarrow d \cdot$ ✓

4
[Item $S' \rightarrow \cdot S$] is accepted.

12. Construct FIRST:-

$S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$

Input

aabb

$S \rightarrow aSb \mid ab$

$FIRST(S) \Rightarrow \{A\}$

$FIRST(A) \Rightarrow \{a\}$

$FIRST(B) \Rightarrow \{b\}$

Parse tree

Parse tree cannot be generate, hence string is not accepted.

13) $S \rightarrow A$

$A \rightarrow aB / Ad$

$B \rightarrow b$

$C \rightarrow q$

$A \rightarrow dA'$

$A' \rightarrow dA' / \epsilon$

$FIRST(S) = \{ A \}$

$FIRST(A) = \{ a, d \}$

$FIRST(A') = \{ d, \epsilon \}$

$FIRST(B) = \{ b \}$

$FIRST(C) = \{ q \}$

$FOLLOW(S) = \{ \$ \}$

$FOLLOW(A) = \{ d, \$ \}$

$FOLLOW(A') = \{ d, \$ \}$

$FOLLOW(B) = \{ d, \$ \}$

$FOLLOW(C) = \{ d, \$ \}$

14) Lex code

```
#include <stdio.h>
```

```
int main () {
```

```
    int a = 10, b = 20;
```

```
    int sum = a + b;
```

```
    printf (" %d ", sum);
```

```
}
```

```
return 0;
```

$S \rightarrow AB$

$A \rightarrow aA \mid a$

$B \rightarrow bB \mid b$

Find

$LR(0)$ items

and LR(0) items

(6)

$S \rightarrow AB$

$A \rightarrow aA \mid \epsilon$

$B \rightarrow bB \mid c$

Compute FOLLOW

$FIRST(S) = \{a, \epsilon\}$

$FIRST(A) = \{a, \epsilon\}$

$FIRST(B) = \{b, c\}$

$FOLLOW(S) = \{\$ \}$

$FOLLOW(A) = \{a, \$ \}$

$FOLLOW(B) = \{ \$ \}$

17) Left Recursion

$$A = \alpha B \beta$$

$$A' = B A'$$

$$A' = \alpha \beta' / \epsilon$$

Left Factoring

18)

$$E \rightarrow T E'$$

$$E' \rightarrow + T E' / \epsilon$$

$$T \rightarrow F T'$$

$$T' \rightarrow * F T' / \epsilon$$

$$F \rightarrow (E) / id$$

Shift Reduce :-

i) Recursive - decrease

ii) Shift reduce Parsing.

Input :-

id + id * id

	Input string	Action.
\$	id + id * id \$	Shift.
\$ id	+ id * id \$	Reduce $F \rightarrow id$.
\$ F	+ id * id \$	Shift
\$ F +	id * id \$	

Recursive descent:

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \Sigma$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \Sigma$$

$$F \rightarrow (E) \mid id.$$

FIRST & FOLLOW

$$FIRST(E) = \{ (, id \}$$

$$FIRST(E') = \{ +, \Sigma \}$$

$$FIRST(T) = \{ (, id \}$$

$$FIRST(T') = \{ *, \Sigma \}$$

$$FIRST(F) = \{ (, id \}$$

$$FOLLOW(E) = \{ \$,) \}$$

$$FOLLOW(E') = \{ +, \$,) \}$$

$$FOLLOW(T) = \{ +, \$,) \}$$

$$FOLLOW(T') = \{ *, \$,) \}$$

$$FOLLOW(F) = \{ +, *, \$,) \}$$

Non terminal	id	(	)	+	*	\$
E	$E \rightarrow TE'$	$E \rightarrow TE'$				
E'				$E' \rightarrow +TE'$		$E' \rightarrow \Sigma$
T	$T \rightarrow FT'$	$T \rightarrow FT'$				
T'			$T' \rightarrow \Sigma$	$T' \rightarrow \Sigma$	$T' \rightarrow *FT'$	$T' \rightarrow \Sigma$
F	$F \rightarrow (id)$	$F \rightarrow (E)$				

Stack Implementation:

	Stack	Action
\$	id + id * id \$	$E \rightarrow TE'$
\$ E' T	id + id * id \$	$T \rightarrow id$
\$ E' T F	id + id * id \$	$F \rightarrow id$
\$ E' T F id	+ id * id \$	$T' \rightarrow \epsilon$
\$ E' T	+ id * id \$	$E' \rightarrow + TE'$
\$ E' T x	* id * id \$	$E' \rightarrow \epsilon$
\$ T	id * id \$	$T \rightarrow FT'$
\$ T F	id * id \$	$F \rightarrow id$
\$ T F id	* id \$	$T' \rightarrow * FT'$
\$ T F id *	* id \$	$F \rightarrow id$
\$ T F id	\$	$T' \rightarrow \epsilon$
\$ F id	\$	Accepted string

Shift Reducing :-

	Input string	Action.
\$	id + id * id \$	shift.
\$ id	+ id * id \$	reduce $E \rightarrow id$
\$ E	+ id * id \$	shift
\$ E +	id * id \$	reduce $E \rightarrow id$
\$ E + E	* id \$	reduce $E \rightarrow E + E$
\$ E	* id \$	shift
\$ E *	id \$	reduce $E \rightarrow id$
\$ E * id	\$	reduce $E \rightarrow E * id$
\$ E * E	\$	Accept
\$	\$	

10

171

 $E \rightarrow E + E$ $E \rightarrow E * E$ $E \rightarrow (E)$

Input

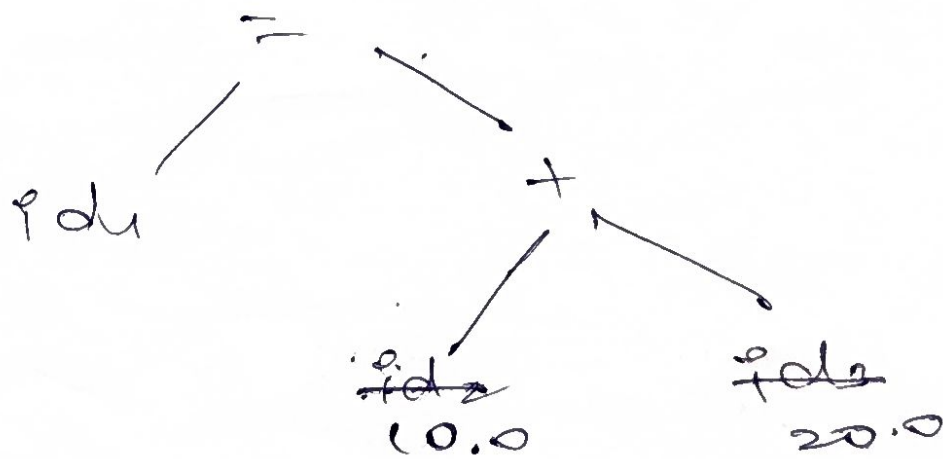
 $E \rightarrow id$ $id + id * id$

	Input string.	Action.
\$	$id + id * id \$$	shift
\$ id	$+ id * id \$$	Reduce $E \rightarrow id$, shift.
\$ E	$+ id * id \$$	shift.
\$ $E +$	$id * id \$$	shift.
\$ $E + id$	$* id \$$	Reduce $E \rightarrow id$
\$ $E + E$	$* id \$$	Reduce $E \rightarrow E + E$
\$ E	$* id \$$	shift
\$ $E *$	$id \$$	shift
\$ $E * id$	$\$$	Reduce $E \rightarrow id$
\$ $E * E$	$\$$	Reduce $E \rightarrow E * E$
\$ E	$\$$	

```
# Include <stdio.h>
int main () {
    int a = 10, b = 20;
    int sum = a + b;
    printf ("%d", sum);
}
```

a) Lexical Analysis

b Syntax :-



19

 $E \rightarrow E + E$ Intermediate code generator $t_2 = \text{int to real } (10.0)$ $t_3 = \text{int to real } (20.0)$ $t_1 = t_2 + t_3$ $pd_1 = t_1$ Code optimizer $t_1 = t_2 + t_3$ $t_1 = pd_1$ code generator :-LDF $R_2, \#10.0$ LDF $R_1, \#20.0$ ADDF R_2, R_1, R_1 STF R_1, id_1